



Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

РОЗРАХУНКОВА РОБОТА З БАЗ ДАНИХ ТА ІНФОРМАЦІЙНИХ СИСТЕМ

Виконала:

студентка групи ФІ-73

Звичайна Анастасія Олександрівна

залікова книжка №04

Київ 2020

Мета: Освоїти методи проектування баз даних і роботи з базами даних у середовищі СУБД MS SQL Server.

Умова:

Предметна область – Деканат.

Задачі, що вирішує інформаційна система – навчальне навантаження викладачів.

Завдання на обробку даних:

1. Створити впорядковані по кафедрам списки:

- Кафедра - викладач - дисципліна;
- Викладачів, у яких посада не відповідає наукового ступеня (для кандидата - доцент, для доктора - професор).

2. Обчислення:

- Переконатися, що у кожного викладача не більше 3-х занять на день.

3. корекція:

- Зміна розкладу для викладача (аудиторія, день тижня, час).

4. Звіти виду:

- Розклад занять для однієї довільної групи.
- "Якісний склад співробітників кафедр": кафедра - кількість професорів - кількість доцентів - кількість старших викладачів;
- Розклад для викладачів кафедри «Х».
- "Навантаження викладачів": кафедра - викладач - кількість дисципліни, які він веде і кількість годин з цих дисциплін

Виконання роботи:

Перше завдання:

Промодельовати функціональні вимоги до системи, що проектується за допомогою DFD-діаграми, для чого:

- Ідентифікувати зовнішні об'єкти, що є постачальниками / приймачами інформації, і види інформації, які циркулюють між системою і зовнішніми об'єктами;
- Визначити основні функціональні вимоги до системи і представити їх у вигляді процесів;
- Визначити основні види інформації і представити їх у вигляді потоків між процесами і зовнішніми об'єктами;
- Задати сховища інформації.

Для побудови DFD-діаграми використовувати інструментальний засіб CASE Studio 2.

Для побудови DFD-діаграми я використовую сайт app.diagrams.net, з яким я познайомилась на третьому курсі при вивченні BPMN позначень. Результати виконання даного завдання представлені на рисунках 1-5. На рисунку 1 подана загальна схема DFD-діаграми, а на рисунках 1-4 підсистеми.

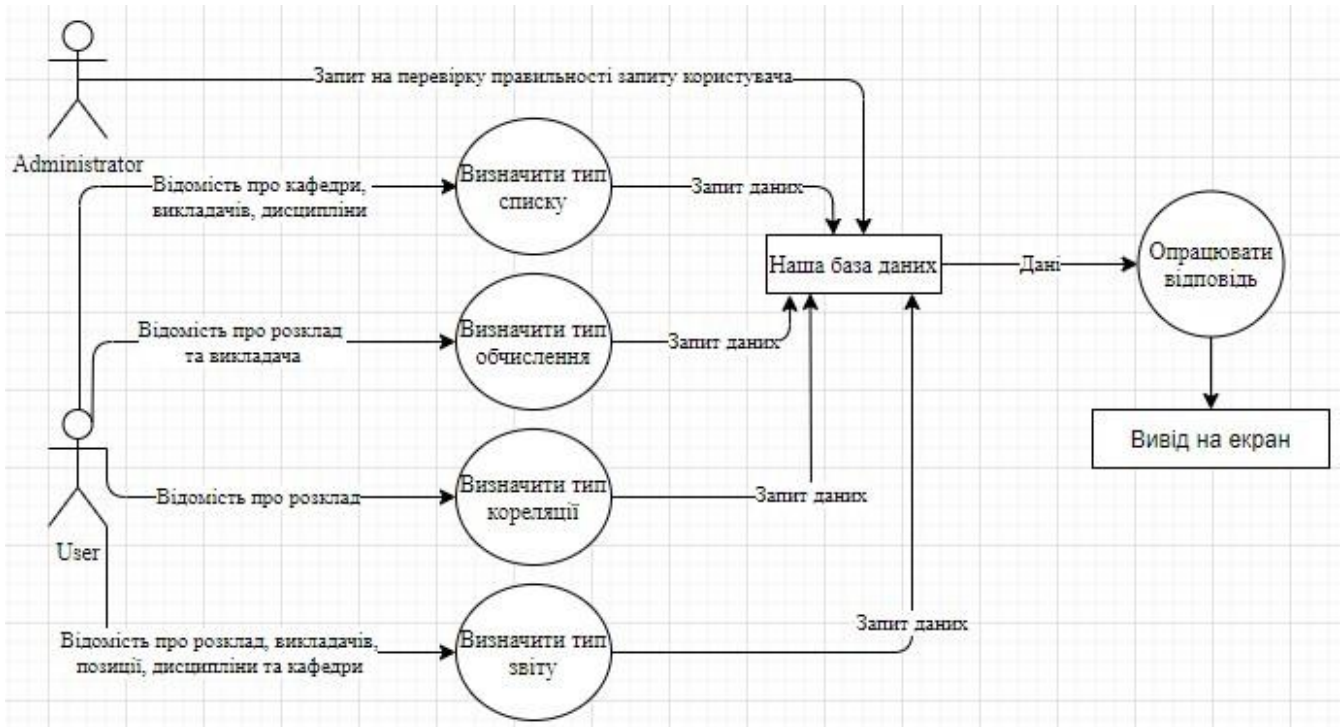


Рис.1. DFD-діаграма.

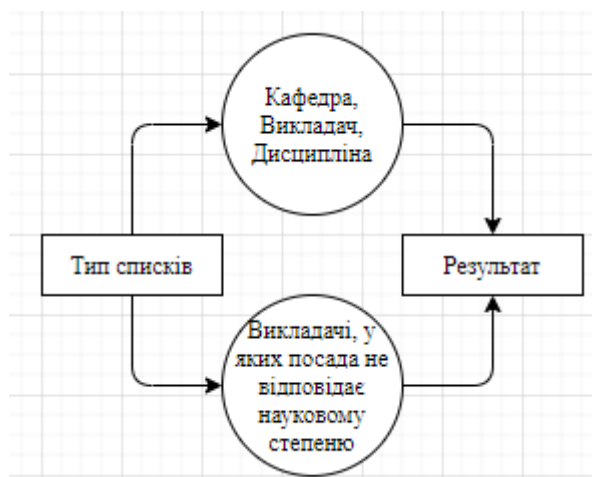


Рис.2. Підсистема типу списків.

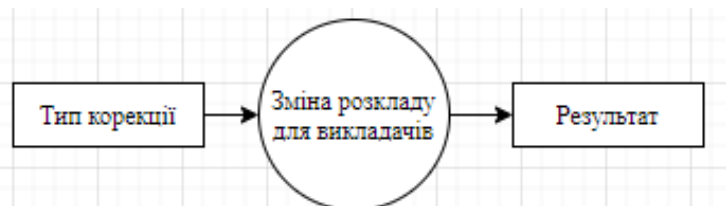


Рис.3. Підсистема типу корекції.

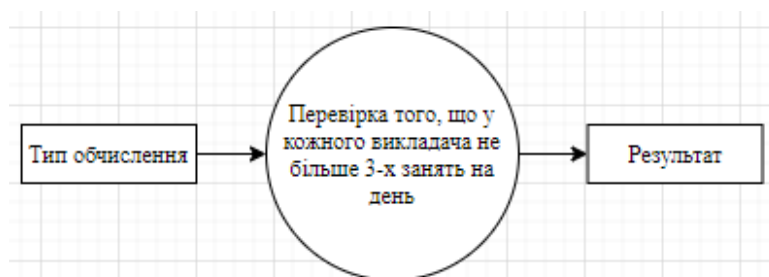


Рис.4. Підсистема типу обчислення.

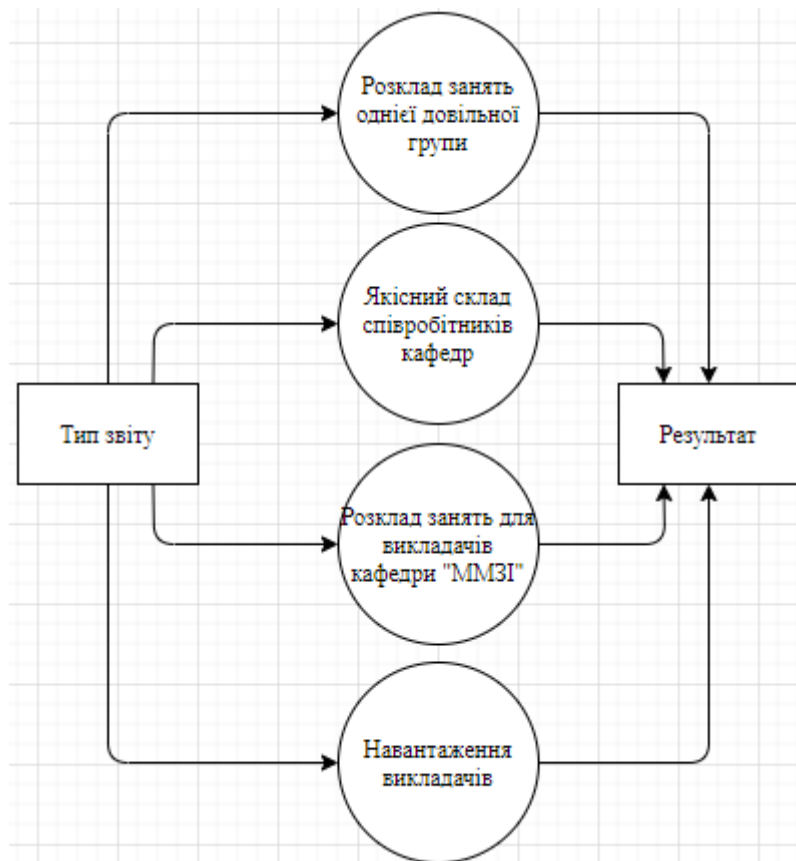


Рис.5. Підсистема тип звіту.

Друге завдання:

Побудувати концептуальну модель (ER-модель), для чого:

- За описом предметної області і поставленому завданню виявити необхідний набір сутностей, їх властивості і зв'язки між сутностями;
- Визначити необхідний набір атрибутів для кожної сутності, виділивши атрибути що ідентифікують сутність;
- Класифікувати сутності (стрижневі, асоціативні та інші);
- Визначити зв'язки між об'єктами;
- Задати обмеження цілісності;
- Описати отриману ER-модель.

Для побудови ER-моделі використовувати інструментальний засіб CASE Studio 2.

Для побудови ER-моделі я також використовую сайт app.diagrams.net. Результат виконання даного завдання можна побачити на рисунку 6, де первинні ключі позначені як РК, а зовнішні ключі позначені як FK.

Сутності: Position, Degree, Discipline, Employees, Department, Department_type, Group_stud, Schedule.

Асоціативні сутності: Department_type (адже пов'язує Employees та Department), Employees (адже пов'язує Position, Degree, Department_type та Schedule).

Стрижневі сутності: Position, Degree, Discipline, Department, Group_stud.

Обмеження: Вік викладача ≥ 24 (вираховуємо різницю між сьогоднішньою датою та датою народження), ПІБ має бути унікальним.

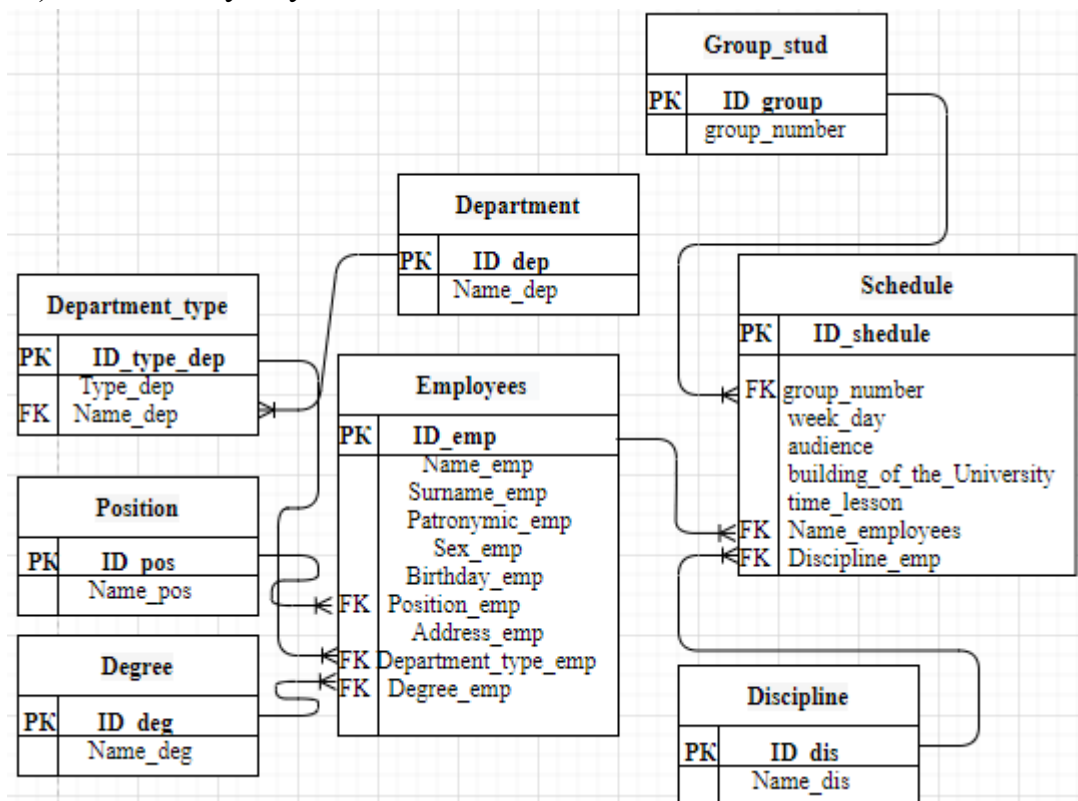


Рис.6. ER-модель, що побудована на сайті app.diagrams.net.

Третє завдання:

Отримати реляційну схему з ER-моделі, для чого:

- Побудувати набір необхідних таблиць бази даних;
- Виділити первинні та зовнішні ключі певних таблиць;
- Привести отримані таблиці до третьої нормальної форми.
- Дати математичний опис отриманої моделі з використанням апарату реляційної алгебри.

Для перевірки нормалізації даної бази даних треба перевірити чи належить вона до третьої нормальної форми. Модель належить до третьої нормальної форми, якщо вона належить до першої та другої нормальної форми, а також не містить транзитивних залежностей.

Дана модель належить до першої нормальної форми, адже всі атрибути атомарні. Також дана модель належить до другої нормальної форми, адже не містить неповних функціональних залежностей не ключових атрибутів від атрибутів первинних ключів. Відповідно дана модель належить до третьої нормальної форми, адже не має транзитивних залежностей та належить до першої та другої нормальної форми.

Четверте завдання:

Використовуючи СУБД MS SQL Server створити спроектовану базу даних.

На рисунку 7 можна побачити ER-схему, що зроблена у середовищі MS SQL.

USE Calculation__work

DROP TABLE IF EXISTS

Schedule,Information,Employees,Discipline,Position,Degree,Group_stud,Department_type,Department;

CREATE TABLE Position(

ID_pos INT IDENTITY PRIMARY KEY NOT NULL,

Name_pos VARCHAR(30) NOT NULL);

CREATE TABLE Discipline(

ID_dis INT IDENTITY PRIMARY KEY NOT NULL,

Name_dis VARCHAR (30) NOT NULL);

CREATE TABLE Department(

ID_dep INT IDENTITY PRIMARY KEY NOT NULL,

Name_dep VARCHAR (50) NOT NULL);

CREATE TABLE Department_type(

ID_type_dep INT IDENTITY PRIMARY KEY NOT NULL,

Type_dep VARCHAR (50) NOT NULL,

Name_dep INT FOREIGN KEY REFERENCES Department(ID_dep) ON DELETE CASCADE);

CREATE TABLE Degree(

ID_deg INT IDENTITY PRIMARY KEY NOT NULL,

Name_deg VARCHAR(30) NOT NULL);

CREATE TABLE Employees(

ID_emp INT IDENTITY PRIMARY KEY NOT NULL,

Name_emp VARCHAR (30) NOT NULL,

Surname_emp VARCHAR (30) NOT NULL,

Patronymic_emp VARCHAR (30) NOT NULL,

UNIQUE(Name_emp,Surname_emp,Patronymic_emp),

Sex_emp VARCHAR (10),

Birthday_emp DATE,

CONSTRAINT Birthday_emp CHECK (

YEAR(GETDATE()) - YEAR(Birthday_emp) > 24 OR

YEAR(GETDATE()) - YEAR(Birthday_emp) = 24 AND MONTH(GETDATE()) > MONTH(Birthday_emp)

OR

YEAR(GETDATE()) - YEAR(Birthday_emp) = 24

AND MONTH(GETDATE()) = MONTH(Birthday_emp)

AND DAY(GETDATE()) >= DAY(Birthday_emp)),

Address_emp VARCHAR (40),

Department_type_emp INT FOREIGN KEY REFERENCES Department_type(ID_type_dep) ON DELETE CASCADE,

```

Position_emp INT FOREIGN KEY REFERENCES Position(ID_pos) ON DELETE CASCADE,
Degree_emp INT FOREIGN KEY REFERENCES Degree(ID_deg) ON DELETE CASCADE);
CREATE TABLE Group_stud(
ID_group INT IDENTITY PRIMARY KEY NOT NULL,
group_number VARCHAR (50) NOT NULL);
CREATE TABLE Schedule(
ID_shedule INT IDENTITY PRIMARY KEY NOT NULL,
group_number INT FOREIGN KEY REFERENCES Group_stud(ID_group) ON DELETE CASCADE,
week_day VARCHAR (50) NOT NULL,
audience INT,
building_of_the_University INT,
time_lesson INT,
Discipline_emp INT FOREIGN KEY REFERENCES Discipline(ID_dis) ON DELETE CASCADE,
Name_employees INT FOREIGN KEY REFERENCES Employees(ID_emp));

```

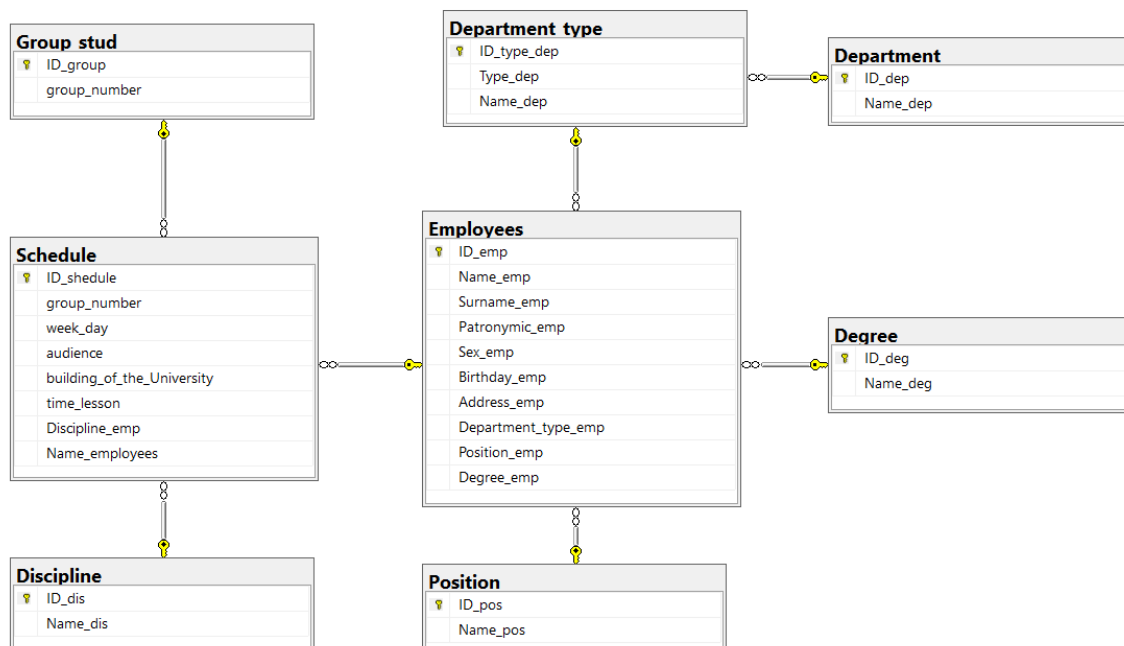


Рис.7.ER-модель, що зроблена у середовищі MS SQL.

П'яте завдання:

На мові SQL записати вирази для зазначених у варіанті завдання типів запитів. Перевірити працездатність написаних запитів в інтерактивному режимі.

Для виконання завдань даної розрахункової роботи спочатку треба було заповнити базу даних значеннями, адже вона була порожньою. Це я виконую аналогічно до 3ї лаби.

```
INSERT INTO Position VALUES ('Доцент'),('Професор'),('Аспірант'),('Асистент'),('Старший викладач');
```

```
INSERT INTO Degree VALUES ('Кандидат наук (PhD)'),('Доктор наук');
```

```
INSERT INTO Discipline VALUES ('Фізика'),
('Криптографія'),('Біологія'),('Хімія'),('Програмування'),('Право'),
```

```

('Соціологія'), ('Психологія'), ('Математичний аналіз'),
('Числові алгоритми'), ('Алгебра та геометрія'), ('Фізкультура');
INSERT INTO Group_stud VALUES ('ФІ-73'), ('ФБ-82'), ('ФФ-82'), ('ФФ-52'), ('БМ-41'), ('БМ-42');
INSERT INTO Department (Name_dep) VALUES ('ФІІ'), ('БМІ');
INSERT INTO Department_type VALUES ('Фізики
наноматеріалів', 1), ('ММЗІ', 1), ('Кібербезпека', 1), ('Біологія', 2);
INSERT INTO Employees
(Name_emp, Surname_emp, Patronymic_emp, Sex_emp, Birthday_emp, Address_emp, Department_type_emp, Position_emp,
Degree_emp)
VALUES ('Віктор', 'Кучеренко', 'Сергійович', 'Ч', '1990-08-20', 'Степана Бандери
7а', 1, 5, 1), ('Петро', 'Брик', 'Семенович', 'Ч', '1995-09-20', 'Перемоги 10а', 2, 3, 1),
('Аліна', 'Чуб', 'Степановна', 'Ж', '1970-04-02', 'Перемоги 10б', 2, 1, 1),
('Катерина', 'Чубенко', 'Віталіївна', 'Ч', '1969-09-11', 'Віноградарь 24г', 3, 1, 1),
('Назарій', 'Жоб', 'Андрійович', 'Ч', '1975-03-01', 'Воздвиженська 20а', 2, 1, 1),
('Степан', 'Чубенко', 'Сергійович', 'Ч', '1966-11-12', 'Віноградарь 24г', 2, 1, 1),
('Дмитро', 'Гетьман', 'Сергійович', 'Ч', '1978-12-13', 'Васильківська 32а', 3, 5, 1),
('Еліна', 'Васильківська', 'Олександрівна', 'Ж', '1988-01-03', 'Степана Бандери 7в', 2, 5, 1),
('Дарина', 'Кузьменко', 'Віталіївна', 'Ж', '1993-06-06', 'Васильківська 31б', 4, 3, 1),
('Максим', 'Деркач', 'Сергійович', 'Ч', '1992-05-03', 'Віноградарь 25б', 3, 3, 1),
('Вікторія', 'Заїка', 'Павлівна', 'Ж', '1938-09-09', 'Перемоги 34д', 2, 2, 2),
('Валерія', 'Шевченко', 'Миколаївна', 'Ж', '1958-04-17', 'Кирилівська 19а', 1, 2, 2),
('Олександр', 'Авраменко', 'Миколайович', 'Ч', '1963-04-22', 'Воздвиженська 18а', 4, 2, 2),
('Марія', 'Замкова', 'Олександрівна', 'Ж', '1987-10-07', 'Лєскова 19б', 2, 4, 1),
('Владислав', 'Соколовський', 'Геннадійович', 'Ч', '1968-08-09', 'Воздвиженська 21а', 1, 2, 1),
('Валентина', 'Шуфра', 'Вікторівна', 'Ж', '1989-08-06', 'Лєскова 19в', 1, 4, 1),
('Валентина', 'Куст', 'Степанівна', 'Ж', '1973-07-02', 'Віноградарь 19в', 4, 5, 1),
('Катерина', 'Морозюк', 'Геннадіївна', 'Ж', '1987-11-02', 'Кирилівська 20б', 4, 5, 1),
('Миколай', 'Буряк', 'Андрійович', 'Ч', '1942-04-05', 'Перемоги 17г', 4, 2, 2);
INSERT INTO Schedule
(group_number, week_day, audience, building_of_the_University, time_lesson, Discipline_emp, Name_employees)
VALUES (1, 'Понеділок', 123, 7, 4, 1, 1), (1, 'Вівторок', 46, 16, 1, 2, 2), (1, 'Четверг', 103, 11, 3, 2,
8),
(1, 'Середа', 103, 11, 2, 5, 10), (1, 'Середа', 103, 11, 3, 9, 4), (1, 'Четверг', 103, 11, 4, 11, 12),
(2, 'Понеділок', 107, 7, 4, 11, 15), (2, 'Середа', 48, 16, 3, 2, 8), (2, 'Середа', 48, 16, 4, 10, 8),
(3, 'Середа', 123, 1, 1, 11, 5), (3, 'Четверг', 106, 11, 2, 5, 6), (3, 'Четверг', 106, 11, 3, 5, 7),
(4, 'Вівторок', 101, 11, 1, 12, 9), (4, 'Вівторок', 101, 11, 2, 10, 7), (4, 'Понеділок', 103, 11, 2, 7,
13),
(5, 'Вівторок', 132, 13, 1, 3, 16), (5, 'Вівторок', 132, 13, 2, 4, 17), (6, 'Четверг', 133, 13, 1, 12,
16),
(6, 'Четверг', 133, 13, 2, 3, 18), (6, 'Четверг', 139, 13, 3, 4, 1);

```

1. Створити впорядковані по кафедрам списки:

- Кафедра - викладач - дисципліна;

- Викладачів, у яких посада не відповідає наукового ступеня (для кандидата - доцент, для доктора - професор).


```

GROUP BY S.week_day, E.ID_emp
        HAVING COUNT(*) > 3)
        PRINT ('Наявний викладач, який має більше 3 пар.')
ELSE PRINT('Немає викладачів, які мають більше 3 пар у день.')
GO

```

У основному запиті робимо виклик даної процедури:

```
EXEC check_the_number_of_lessons
```

Вивід на екран можна побачити на рисунку 9.

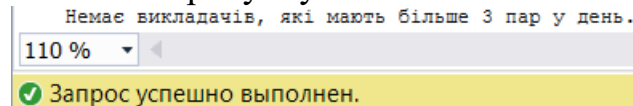


Рис.9. Результат виконання запиту завдання 2.

3. корекція:

- Зміна розкладу для викладача (аудиторія, день тижня, час).

Дане завдання також виконуємо у вигляді процедури, адже її можна потім використовувати як окремий додаток.

У окремому запит-файлі пишемо наступну процедуру:

```

USE Calculation_work

GO

--Зміна розкладу для викладача (аудиторія, день тижня, час).
CREATE PROC change_schedule
@schedule_id INT OUTPUT,
@new_week_day VARCHAR(50) OUTPUT,
@new_audience INT OUTPUT,
@new_building_of_the_University INT OUTPUT,
@new_time_lesson INT OUTPUT
AS
BEGIN
UPDATE Schedule SET week_day = @new_week_day,
                    audience=@new_audience,
                    building_of_the_University=@new_building_of_the_University,
                    time_lesson=@new_time_lesson
WHERE ID_shedule = @schedule_id
END;
GO

```

У основному запиті робимо виклик даної процедури:

```

EXEC change_schedule 1, 'Середа', 105, 11, 1
SELECT G.group_number, S.week_day, S.audience, S.building_of_the_University, S.time_lesson, D.Name_dis,
E.Name_emp, E.Surname_emp, E.Patronymic_emp FROM Schedule S, Group_stud G, Discipline D, Employees E
WHERE G.ID_group=S.group_number AND S.Discipline_emp=D.ID_dis AND E.ID_emp=S.Name_employees

```

Виконуючи запит **SELECT** на рисунку 10 бачимо, що розклад змінився.

Результаты Сообщения									
	group_number	week_day	audience	building_of_the_University	time_lesson	Name_dis	Name_emp	Surname_emp	Patronymic_emp
1	ФІ-73	Середа	105	11	1	Фізика	Віктор	Кучеренко	Сергійович
2	ФІ-73	Вівторок	46	16	1	Криптографія	Петро	Брик	Семенович
3	ФІ-73	Четверг	103	11	3	Криптографія	Еліна	Васильківська	Олександрівна
4	ФІ-73	Середа	103	11	2	Програмування	Максим	Деркач	Сергійович
5	ФІ-73	Середа	103	11	3	Математичний аналіз	Катерина	Чубенко	Віталіївна
6	ФІ-73	Четверг	103	11	4	Алгебра та геометрія	Валерія	Шевченко	Миколаївна
7	ФБ-82	Понеділок	107	7	4	Алгебра та геометрія	Владислав	Соколовський	Геннадійович
8	ФБ-82	Середа	48	16	3	Криптографія	Еліна	Васильківська	Олександрівна
9	ФБ-82	Середа	48	16	4	Числові алгоритми	Еліна	Васильківська	Олександрівна
10	ФФ-82	Середа	123	1	1	Алгебра та геометрія	Назарій	Жоб	Андрійович
11	ФФ-82	Четверг	106	11	2	Програмування	Степан	Чубенко	Сергійович
12	ФФ-82	Четверг	106	11	3	Програмування	Дмитро	Гетьман	Сергійович
13	ФФ-52	Вівторок	101	11	1	Фізкультура	Дарина	Кузьменко	Віталіївна
14	ФФ-52	Вівторок	101	11	2	Числові алгоритми	Дмитро	Гетьман	Сергійович
15	ФФ-52	Понеділок	103	11	2	Соціологія	Олександр	Авраменко	Миколайович
16	БМ-41	Вівторок	132	13	1	Біологія	Валентина	Шустрі	Вікторівна
17	БМ-41	Вівторок	132	13	2	Хімія	Валентина	Куст	Степанівна
18	БМ-42	Четверг	133	13	1	Фізкультура	Валентина	Шустрі	Вікторівна
19	БМ-42	Четверг	133	13	2	Біологія	Катерина	Морозюк	Геннадіївна
20	БМ-42	Четверг	139	13	3	Хімія	Віктор	Кучеренко	Сергійович

Рис.10. Результат виконання запиту завдання 3.

4. Звіти виду:

- Розклад занять для однієї довільної групи.
- "Якісний склад співробітників кафедр": кафедра - кількість професорів - кількість доцентів - кількість старших викладачів;
- Розклад для викладачів кафедри «ММЗІ».
- "Навантаження викладачів": кафедра - викладач - кількість дисципліни, які він веде і кількість годин з цих дисциплін

У окремому запит-файлі пишемо процедуру, що виводить розклад занять для певної групи:

```

USE Calculation_work

GO

-- Розклад занять для однієї довільної групи.

CREATE PROC schedule_for_one_group
@group_num VARCHAR(50) OUTPUT
AS
SELECT
S.week_day,S.audience,S.building_of_the_University,S.time_lesson,Disc.Name_dis,E.Surname_emp,E.Name_emp
,E.Patronymic_emp
FROM Schedule S,Employees E,Discipline Disc,Group_stud G
WHERE E.ID_emp=S.Name_employees AND S.Discipline_emp=Disc.ID_dis AND G.ID_group=S.group_number AND
G.group_number=@group_num

GO

```

У основному запиті робимо виклик даної процедури для групи ФІ-73:

```
EXEC schedule_for_one_group 'ФІ-73'
```

Результат виконання такого запиту можна побачити на рисунку 11.

Результаты		Сообщения						
	week_day	audience	building_of_the_University	time_lesson	Name_dis	Surname_emp	Name_emp	Patronymic_emp
1	Понеділок	123	7	4	Фізика	Кучеренко	Віктор	Сергійович
2	Вівторок	46	16	1	Криптографія	Брик	Петро	Семенович
3	Четверг	103	11	3	Криптографія	Васильківська	Еліна	Олександрівна
4	Середа	103	11	2	Програмування	Деркач	Максим	Сергійович
5	Середа	103	11	3	Математичний аналіз	Чубенко	Катерина	Віталіївна
6	Четверг	103	11	4	Алгебра та геометрія	Шевченко	Валерія	Миколаївна

Запрос успешно выполнен. DESKTOP-6SF01AQ\SQL_EXE

Рис.11.Результат виконання запиту завдання 4.1.

У моїй базі даних є не тільки професори та старші викладачі, тому я вважаю корисним вивести інформацію і про кількість інших позицій (асистентів, доцентів, аспірантів).

У окремому запит-файлі пишемо наступну процедуру:

```
USE Calculation_work

GO

-- "Якісний склад співробітників факультетів/департаментів/інститутів": кафедра-кількість професорів-
кількість доцентів-кількість старших викладачів.

CREATE PROC qualitative_staff_of_departments

AS

SELECT Name_dep,Name_pos, COUNT(*) as Amount FROM

(SELECT DISTINCT D.Name_dep,P.Name_pos,E.ID_emp FROM Employees E,Position P,Department D,
Department_type T

WHERE E.Department_type_emp=T.ID_type_dep AND T.Name_dep=D.ID_dep AND E.Position_emp = P.ID_pos) AS
help_table

GROUP BY Name_dep, Name_pos

ORDER BY Name_dep

GO
```

У основному запиті робимо виклик даної процедури:

```
EXEC qualitative_staff_of_departments
```

Результат виконання такого запиту можна побачити на рисунку 12.a.

Результаты		Сообщения	
	Name_dep	Name_pos	Amount
1	БМІ	Аспірант	1
2	БМІ	Професор	2
3	БМІ	Старший викладач	2
4	ФТІ	Асистент	2
5	ФТІ	Аспірант	2
6	ФТІ	Доцент	4
7	ФТІ	Професор	3
8	ФТІ	Старший викладач	3

Запрос успешно выполнен.

Рис.12.Результат виконання запиту завдання 4.2.a.

У окремому запит-файлі пишемо наступну процедуру:

```
USE Calculation_work
```

```
GO
```

```
-- "Якісний склад співробітників кафедр": кафедра - кількість професорів - кількість доцентів - кількість старших викладачів.
```

```
CREATE PROC qualitative_staff_of_type_departments
```

```
AS
```

```
SELECT Type_dep, Name_pos, COUNT(*) as Amount FROM
```

```
(SELECT DISTINCT T.Type_dep, P.Name_pos, E.ID_emp FROM Employees E, Position P, Department_type T
```

```
WHERE E.Department_type_emp=T.ID_type_dep AND E.Position_emp = P.ID_pos) AS help_table
```

```
GROUP BY Type_dep, Name_pos
```

```
ORDER BY Type_dep
```

```
GO
```

У основному запиті робимо виклик даної процедури:

```
EXEC qualitative_staff_of_type_departments
```

Результаты		Сообщения	
	Type_dep	Name_pos	Amount
1	Біологія	Аспірант	1
2	Біологія	Професор	2
3	Біологія	Старший викладач	2
4	Кибербезопаска	Аспірант	1
5	Кибербезопаска	Доцент	1
6	Кибербезопаска	Старший викладач	1
7	ММЗІ	Асистент	1
8	ММЗІ	Аспірант	1
9	ММЗІ	Доцент	3
10	ММЗІ	Професор	1
11	ММЗІ	Старший викладач	1
12	Фізики нано...	Асистент	1
13	Фізики нано...	Професор	2
14	Фізики нано...	Старший викладач	1

Рис.12.Результат виконання запиту завдання 4.2.б.

У окремому запит-файлі пишемо процедуру, що виводить розклад для викладачів певної кафедри:

```
USE Calculation_work
```

```
GO
```

```
-- Розклад для викладачів певної кафедри.
```

```
CREATE PROC schedule_employees_of_one_department
```

```
@dep_name VARCHAR (30) OUTPUT
```

```
AS
```

```
SELECT
```

```
S.week_day, S.audience, S.building_of_the_University, S.time_lesson, Disc.Name_dis, E.Name_emp, E.Surname_emp  
, E.Patronymic_emp
```

```
FROM Schedule S, Employees E, Department_type T, Discipline Disc
```

```
WHERE @dep_name=T.Type_dep AND S.Name_employees = E.ID_emp AND E.Department_type_emp = T.ID_type_dep
```

```
AND Disc.ID_dis=S.Discipline_emp
```

```
GO
```

У основному запиті робимо виклик даної процедури для кафедри ММЗІ:

EXEC schedule_employees_of_one_department 'ММЗІ'

Результат виконання такого запиту можна побачити на рисунку 13.

Результаты		Сообщения						
	week_day	audience	building_of_the_University	time_lesson	Name_dis	Name_emp	Surname_emp	Patronymic_emp
1	Вівторок	46	16	1	Криптографія	Петро	Брик	Семенович
2	Четверг	103	11	3	Криптографія	Еліна	Васильківська	Олександрівна
3	Середа	48	16	3	Криптографія	Еліна	Васильківська	Олександрівна
4	Середа	48	16	4	Числові алгоритми	Еліна	Васильківська	Олександрівна
5	Середа	123	1	1	Алгебра та геометрія	Назарій	Жоб	Андрійович
6	Четверг	106	11	2	Програмування	Степан	Чубенко	Сергійович

Рис.13. Результат виконання запиту завдання 4.3.

Зазначу, що пара триває 1 годину та 45 хвилин, що те саме, що 1.75 годин.

У окремому запит-файлі пишемо процедуру, що звітує про навантаження викладачів:

USE Calculation_work

GO

-- "Навантаження викладачів": кафедра-викладач-кількість дисципліни, які він веде і кількість годин з цих дисциплін.

CREATE PROC workload_of_employees

AS

SELECT Dep.Type_dep, E.Name_emp, E.Surname_emp, E.Patronymic_emp, help_table.amount_of_discipline, 1.75 * COUNT (Disc.ID_dis) as number_of_hours

FROM Employees E, Discipline Disc, Schedule S, Department_type Dep,

(SELECT Schedule.Name_employees, COUNT(Schedule.Discipline_emp) AS amount_of_discipline FROM Schedule, Employees

WHERE Schedule.Name_employees = Employees.ID_emp

GROUP BY Schedule.Name_employees) AS help_table

WHERE E.ID_emp=S.Name_employees AND S.Discipline_emp=Disc.ID_dis

AND E.Department_type_emp=Dep.ID_type_dep AND help_table.Name_employees=E.ID_emp

GROUP BY Type_dep, Name_emp, Surname_emp, Patronymic_emp, amount_of_discipline

GO

У основному запиті робимо виклик даної процедури:

EXEC workload_of_employees

Результат виконання такого запиту можна побачити на рисунку 14.

Результаты		Сообщения				
	Type_dep	Name_emp	Surname_emp	Patronymic_emp	amount_of_discipline	number_of_hours
1	Біологія	Валентина	Куст	Степанівна	1	1.75
2	Біологія	Дарина	Кузьменко	Віталіївна	1	1.75
3	Біологія	Катерина	Морозюк	Геннадійовна	1	1.75
4	Біологія	Олександр	Авраменко	Миколайович	1	1.75
5	Кібербезпека	Дмитро	Гетьман	Сергійович	2	3.50
6	Кібербезпека	Катерина	Чубенко	Віталіївна	1	1.75
7	Кібербезпека	Максим	Деркач	Сергійович	1	1.75
8	ММЗІ	Еліна	Васильківська	Олександрівна	3	5.25
9	ММЗІ	Назарій	Жоб	Андрійович	1	1.75
10	ММЗІ	Петро	Брик	Семенович	1	1.75
11	ММЗІ	Степан	Чубенко	Сергійович	1	1.75
12	Фізики наноматеріалів	Валентина	Шустра	Вікторівна	2	3.50
13	Фізики наноматеріалів	Валерія	Шевченко	Миколаївна	1	1.75
14	Фізики наноматеріалів	Віктор	Кучеренко	Сергійович	2	3.50
15	Фізики наноматеріалів	Владислав	Соколовський	Геннадійович	1	1.75

Запрос успешно выполнен. DESKTOP

Рис.14. Результат виконання запиту завдання 4.4.

Шосте завдання:

Реалізувати закінчене додаток, що працює з створеною базою даних.

Додаток має

- Заносити інформацію в таблиці створеної бази даних,;
- Виконувати необхідні дії по модифікації і видалення даних в таблицях створеної бази даних;
- Підтримувати цілісність бази даних;
- Виконувати запити з варіанта завдання.

Створимо процедури, що заносять данні у таблиці:

```
USE Calculation_work
```

```
GO
```

```
--Створимо процедури, що заносять дані у таблиці
```

```
CREATE PROC add_Employees
```

```
@Name_emp      VARCHAR (30) OUTPUT,
```

```
@Surname_emp    VARCHAR (30) OUTPUT,
```

```
@Patronymic_emp VARCHAR (30) OUTPUT,
```

```
@Sex_emp        VARCHAR (10) OUTPUT,
```

```
@Birthday_emp   DATE OUTPUT,
```

```
@Address_emp    VARCHAR (40) OUTPUT,
```

```
@Department_type_emp INT OUTPUT,
```

```
@Position_emp   INT OUTPUT,
```

```
@Degree_emp     INT OUTPUT
```

```
AS
```

```
IF NOT EXISTS(SELECT Name_emp,Surname_emp,Patronymic_emp FROM Employees --цього порівняння достатньо, адже ПІБ унікальне
```

```
WHERE Name_emp=@Name_emp AND @Surname_emp=Surname_emp AND @Patronymic_emp=Patronymic_emp)
```

```
INSERT INTO Employees
```

```
(Name_emp,Surname_emp,Patronymic_emp,Sex_emp,Birthday_emp,Address_emp,Department_type_emp,Position_emp,Degree_emp)
```

```
VALUES
```

```
(@Name_emp,@Surname_emp,@Patronymic_emp,@Sex_emp,@Birthday_emp,@Address_emp,@Department_type_emp,@Position_emp,@Degree_emp)
```

```
GO
```

```
CREATE PROC add_Schedule
```

```
@group_number  VARCHAR (50) OUTPUT,
```

```
@week_day      VARCHAR (50) OUTPUT,
```

```
@audience     INT OUTPUT,
```

```
@building_of_the_University INT OUTPUT,
```

```
@time_lesson   INT OUTPUT,
```

```

@Discipline_emp INT OUTPUT,

@Name_employees INT OUTPUT

AS

IF NOT EXISTS(SELECT
group_number,week_day,audience,building_of_the_University,time_lesson,Discipline_emp FROM Schedule --
викладача не перевіряємо, адже він може змінюватись, якщо розклад тільки встановлюється

WHERE @group_number=group_number AND @week_day=week_day AND @audience=audience AND
@building_of_the_University=@building_of_the_University AND @time_lesson=time_lesson AND
@Discipline_emp=Discipline_emp)

    INSERT INTO Schedule
(group_number,week_day,audience,building_of_the_University,time_lesson,Discipline_emp,Name_employees)

    VALUES
(@group_number,@week_day,@audience,@building_of_the_University,@time_lesson,@Discipline_emp,@Name_employees)

GO

CREATE PROC add_Discipline

@Name_dis VARCHAR (30) OUTPUT

AS

IF NOT EXISTS(SELECT Name_dis FROM Discipline

WHERE Name_dis=@Name_dis)

    INSERT INTO Discipline VALUES (@Name_dis)

GO

--Ступені не додаємо, адже їх існує тільки дві

CREATE PROC add_Position

@Name_pos VARCHAR(30) OUTPUT

AS

IF NOT EXISTS(SELECT Name_pos FROM Position

WHERE Name_pos=@Name_pos)

    INSERT INTO Position VALUES (@Name_pos)

GO

CREATE PROC add_Department

@Name_dep VARCHAR(30) OUTPUT

AS

IF NOT EXISTS(SELECT Name_dep FROM Department

WHERE @Name_dep=Name_dep)

    INSERT INTO Department VALUES (@Name_dep)

GO

CREATE PROC add_Department_type

@Type_dep VARCHAR (30) OUTPUT,

@Name_dep INT OUTPUT

```

```

AS
IF NOT EXISTS(SELECT Type_dep,Name_dep FROM Department_type
WHERE @Name_dep=Name_dep AND Type_dep=@Type_dep)
    INSERT INTO Department_type VALUES (@Type_dep,@Name_dep)

```

```

GO

CREATE PROC add_Group_stud
@group_number VARCHAR (50) OUTPUT

```

```

AS
IF NOT EXISTS(SELECT group_number FROM Group_stud
WHERE @group_number=group_number)
    INSERT INTO Department VALUES (@group_number)

```

```

GO

```

Створимо процедури, що видаляють данні з таблиць:

```

USE Calculation__work

```

```

GO

```

```

--Створимо процедури, що видаляють дані з таблиць

```

```

CREATE PROC del_Employees

```

```

@ID_emp INT OUTPUT

```

```

AS

```

```

IF EXISTS(SELECT ID_emp FROM Employees --перевірка на існування запису, адже інакше ми не зрозуміло що
видаляємо

```

```

WHERE ID_emp=@ID_emp)

```

```

    DELETE FROM Employees

```

```

    WHERE @ID_emp=ID_emp

```

```

GO

```

```

CREATE PROC del_Group_stud

```

```

@ID_group INT OUTPUT

```

```

AS

```

```

IF EXISTS(SELECT ID_group FROM Group_stud --перевірка на існування запису, адже інакше ми не зрозуміло
що видаляємо

```

```

WHERE ID_group=@ID_group)

```

```

    DELETE FROM Group_stud

```

```

    WHERE @ID_group=ID_group

```

```

GO

```

```

CREATE PROC del_Schedule

```

```

@ID_shedule INT OUTPUT

```

```

AS

```

```

IF EXISTS(SELECT ID_shedule FROM Schedule --перевірка на існування запису, адже інакше ми не зрозуміло

```

що видаляємо

```
WHERE @ID_shedule=ID_shedule)
```

```
DELETE FROM Schedule
```

```
WHERE @ID_shedule=ID_shedule
```

```
GO
```

```
CREATE PROC del_Discipline
```

```
@ID_dis INT OUTPUT
```

```
AS
```

```
IF EXISTS(SELECT ID_dis FROM Discipline --перевірка на існування запису, адже інакше ми не зрозуміло що  
видаляємо
```

```
WHERE @ID_dis=ID_dis)
```

```
DELETE FROM Discipline
```

```
WHERE @ID_dis=ID_dis
```

```
GO
```

```
CREATE PROC del_Position
```

```
@ID_pos INT OUTPUT
```

```
AS
```

```
IF EXISTS(SELECT ID_pos FROM Position --перевірка на існування запису, адже інакше ми не зрозуміло що  
видаляємо
```

```
WHERE @ID_pos=ID_pos)
```

```
DELETE FROM Position
```

```
WHERE @ID_pos=ID_pos
```

```
GO
```

```
CREATE PROC del_Department
```

```
@ID_dep INT OUTPUT
```

```
AS
```

```
IF EXISTS(SELECT ID_dep FROM Department --перевірка на існування запису, адже інакше ми не зрозуміло що  
видаляємо
```

```
WHERE @ID_dep=ID_dep)
```

```
DELETE FROM Department
```

```
WHERE @ID_dep=ID_dep
```

```
GO
```

```
CREATE PROC del_Department_type
```

```
@ID_type_dep INT OUTPUT
```

```
AS
```

```
IF EXISTS(SELECT ID_type_dep FROM Department_type --перевірка на існування запису, адже інакше ми не  
зрозуміло що видаляємо
```

```
WHERE @ID_type_dep=ID_type_dep)
```

```
DELETE FROM Department_type
```

```
WHERE @ID_type_dep=ID_type_dep
```

```
GO
```

Цілісність даної бази даних була закладена у 4 завданні: були задані первинні та зовнішні ключі з каскадним видаленням всіх пов'язаних значень; введено обмеження за віком (через дату народження), адже щоб викладати якийсь предмет, треба бути хоча б аспірантом, а ним стають приблизно у 24 роки; реалізована перевірка унікальності ПІБ.

Запити з варіанта завдань були продемонстровані у завданні 5 в інтерактивному режимі.